

Ejercicio 1

Dado el siguiente programa y el análisis de *Very Busy Expressions*:

```
1 int compute(int n) {
2   int a = 0;
3   int b = n;
4   int c = 1;
5   while (b > 0) {
6     int t = a + c;
7     if (b % 2 == 0) {
8       a = t + b;
9     } else {
10      c = t - b;
11    }
12    b = b - 1;
13  }
14  int result = (a + c) * n;
15  return result;
16 }
```

- Construir el control-flow graph del método `compute`.
- Identificar las expresiones aritméticas presentes en el código.
- Calcular los conjuntos $IN[n]$ y $OUT[n]$ de cada nodo según el análisis de *Very Busy Expressions*.
- Indicar cuáles son las *very busy expressions* resultantes del análisis en el $IN[n]$ de cada nodo.

Ejercicio 2

Se busca definir un nuevo análisis de dataflow que permita predecir posibles **valores constantes** que puede tomar una variable entera durante la ejecución de un programa. Por ejemplo, en el siguiente fragmento, se podría inferir que al final de la ejecución x vale 0 y vale 5:

```
int x, y;
x = 3;
y = x + 2;
x = y - 5;
```

Sin embargo, se desea un análisis más general: no se busca el valor exacto, sino un conjunto de posibles constantes que una variable puede tomar. Por ejemplo, que x puede valer $\{0, 1\}$ o que su valor es desconocido.

a. Se desea determinar si una variable está:

- no inicializada,
- toma exactamente un valor constante,

- puede tomar dos valores distintos, o
- su valor es desconocido (puede tomar cualquier entero).

Dibujar un reticulado C completo que modele estos estados.

- Definir la semántica abstracta para las asignaciones y las operaciones aritméticas de suma y resta sobre los elementos del reticulado.
- ¿Cómo se comportaría la operación de join ($\sqcup$) entre dos estados diferentes de una misma variable? Da un ejemplo concreto de dos valores que se combinan, y el resultado de dicha unión (el ejemplo no puede ser $\top \sqcup \perp$)

Ejercicio 3

Consideremos el siguiente programa:

```

1 static int adjust(int x) {
2     if (x < 0)
3         return x + 10;
4     else
5         return x - 5;
6 }
7
8 static void main(String[] args) {
9     int a, b, c;
10    a = 5;
11    b = adjust(a); // llamada 1
12    x = 20;
13    c = ...just(a); // llamada 2
14    System.out.println(b + c);
15 }
```

Queremos realizar un análisis interprocedural de intervalos que, para cada variable, asocie un intervalo $[min, max]$ con los posibles valores enteros que puede tomar en cada punto del programa.

- Dibujar el control-flow graph para los métodos `adjust` y `main`.
- Calcular el análisis de dataflow interprocedural sin contexto de llamada. Mostrar los intervalos $IN[n]$ y $OUT[n]$ en cada punto del programa.
- Repetir el análisis utilizando contextos de llamada con $k = 1$. Mostrar cómo cambia la precisión de los resultados respecto al caso anterior.

Ejercicio 4

Dado el siguiente programa:

```

1 class Node { Node nxt; }
2
3 void main() {
4     Node a = new Node();
5     Node b = new Node();
6     Node c = new Node();
7     Node d = a;
8     b = c;
9     d.nxt = b;
10    a = b;
11    Node e = d.nxt;
12    e.nxt = a;
13    Node f = new Node();
14    b = f;
15    Node g = b.nxt;
16 }
```

- Escribir las restricciones de unificación que genera el algoritmo de Steensgard para este programa.
- Ejecutar paso a paso las unificaciones, mostrando las clases de equivalencia que se van formando entre variables y objetos (allocation sites).
- Dibujar el Points-to Graph (PTG) final resultante del análisis de Steensgard.